



ZeroWall

A Serverless Zero-Trust API Gateway on AWS

*Never trust, **always verify***

● **Raj Desai**

Monitoring · Demo · Testing

● **Karan Vora**

Security · Authorizer · RBAC

● **Dhruv Shelke**

API · Backend · Data Layer

INTRODUCTION

The Problem with "Inside" Trust

TRADITIONAL PERIMETER**Trust the inside, guard the edge**

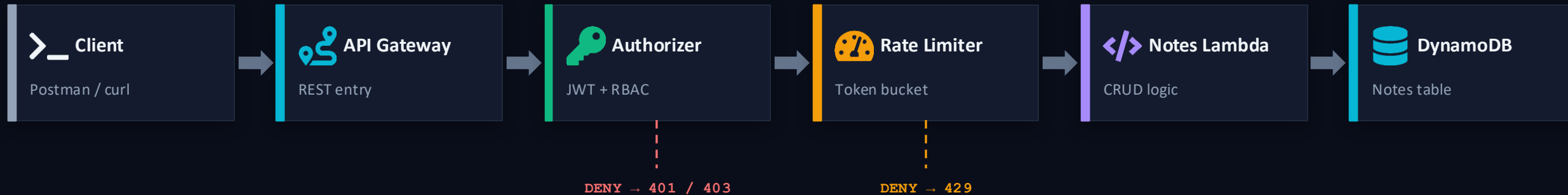
- ✗ Authenticate once at the door
- ✗ Implicit trust for everything inside
- ✗ One breach exposes the whole network
- ✗ Hard to audit what users actually did

ZERO-TRUST APPROACH**Verify every request, every time**

- ✓ Authenticate AND authorize each call
- ✓ Least-privilege access by role
- ✓ Assume breach: defense in depth
- ✓ Full audit trail of every action

Next: *Karan walks through the system design*

System Architecture



Observability layer

Every component logs to CloudWatch · Metrics + Alarms · SNS email alerts on anomalies

The Verification Pipeline

Four independent checks. Every request. No exceptions.

**01**

Authenticate

Who are you?

- JWT extracted from Authorization header
- Signature verified against Cognito JWKS
- Expiry + issuer + audience checked

ON FAIL

401 Unauthorized

**02**

Authorize

What can you do?

- Role extracted from custom:role claim
- Permissions looked up in zerowall-roles
- Method + resource matched against ACL

ON FAIL

403 Forbidden

**03**

Rate Limit

Are you flooding us?

- DynamoDB counter per user, per minute
- Auto-cleaned via TTL on the table
- Threshold enforced before any DB writes

ON FAIL

429 Too Many Requests

**04**

Audit

What just happened?

- Every request logged: allowed or denied
- User, IP, action, latency, outcome captured
- 30-day retention via DynamoDB TTL

ON FAIL

Always logged

Data Model & Access Control

Four DynamoDB Tables

**zerowall-notes**
`KEY    userId + noteId`
User's notes - strict isolation by partition

**zerowall-roles**
`KEY    role`
RBAC permission map: role → allowed actions

**zerowall-rate-limits**
`KEY    userId + windowTimestamp`
Per-minute counters, auto-expire via TTL

**zerowall-audit-log**
`KEY    userId + timestamp`
Full request trail, 30-day TTL

Role Permissions · RBAC

ENDPOINT		USER	ADMIN
GET	/notes	✓	✓
POST	/notes	✓	✓
GET	/notes/{id}	✓	✓
PUT	/notes/{id}	✓	✓
DELETE	/notes/{id}	✗	✓

Data isolation: `userId` always read from the JWT, never from the request body - users cannot reach another user's partition.

Built on AWS Free Tier

Seven managed services, zero servers to maintain.



API Gateway

REST entry point - routes & authorizer attachment



Lambda

Python 3.12 runtime for all backend logic



Cognito

User pool, JWT issuance, custom:role attribute



DynamoDB

4 tables - notes, roles, rate limits, audit



CloudWatch

Logs, metrics, alarms, dashboards



SNS

Email alerts when alarms fire



IAM

Per-Lambda least-privilege execution roles

us-east-2

AWS REGION

Python 3.12

LAMBDA RUNTIME

\$0

FREE TIER COST

100%

SERVERLESS

Four Lambdas, One Pipeline



zerowall-authorizer

TRIGGER API Gateway · REQUEST authorizer

Verifies JWT signature against Cognito JWKS, checks expiry, extracts role, queries roles table, returns IAM Allow/Deny policy

DEPS PyJWT · cryptography · requests



zerowall-notes-handler

TRIGGER API Gateway · proxy integration

Routes CRUD on /notes, runs rate-limit check, enforces userId from authorizer context, writes audit log entries

DEPS boto3 · uuid



zerowall-auth-handler

TRIGGER API Gateway · public endpoints

Wraps Cognito sign-up, login, refresh - sets default role on signup via custom:role attribute

DEPS boto3



zerowall-anomaly-checker

TRIGGER CloudWatch Events · every 5 min

Scans audit log for brute-force patterns, multi-IP usage, and 429 spikes - publishes to SNS

DEPS boto3

Next: [Raj demos the system in action](#)

Authentication & Authorization in Action

Login → JWT issued

200

POST /auth/login

```
{
  "AccessToken": "eyJraWQiOiJ4Vkl...",
  "RefreshToken": "eyJjdHkiOiJKVlQi...",
  "ExpiresIn": 3600,
  "TokenType": "Bearer"
}
```

Create note → allowed

201

POST /notes

```
Authorization: Bearer eyJraWQiOiJ4Vkl...

{
  "noteId": "n-7f3a-c281",
  "title": "First note",
  "createdAt": "2026-04-30T14:22:01Z"
}
```

Missing token → blocked

401

GET /notes

```
{
  "status": "error",
  "error": "UNAUTHORIZED",
  "message": "Token is missing or invalid."
}
```

Delete as user role → blocked

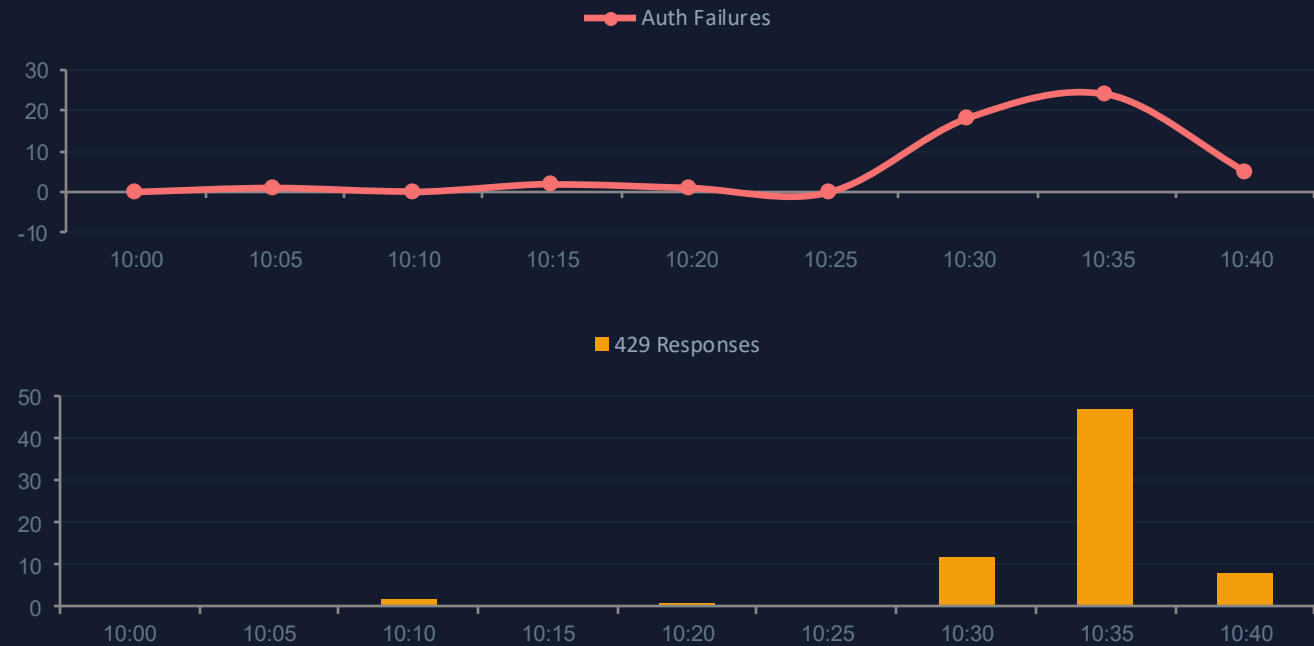
403

DELETE /notes/n-7f3a-c281

```
{
  "status": "error",
  "error": "FORBIDDEN",
  "message": "Role 'user' lacks DELETE on this resource."
}
```


Monitoring, Alerts & Audit Trail

 CloudWatch · zerowall-dashboard LIVE



 **SNS ALERT**
zerowall-alerts → team

ALARM: HighAuthFailures
Threshold: 20 failures / 5 min
Observed: 47 in last 5 min
Top user: u-7f3a (29 fails)
Source IPs: 3 distinct

 **AUDIT LOG**
zerowall-audit-log

10:38:12	POST /notes	ALLOW
10:38:14	DELETE /notes/abc	DENY_RBAC
10:38:18	GET /notes	DENY_RATE
10:38:22	PUT /notes/xyz	ALLOW

DIRECTIONS

Where ZeroWall Goes Next

Six enhancements that move this from coursework to production-grade.



SECURITY

AWS WAF

Edge protection against OWASP Top 10. Block SQL injection, XSS, and bot traffic at API Gateway.



SECURITY

GuardDuty + Security Hub

AWS-native threat detection across CloudTrail, VPC flow logs, and DNS. Centralized findings in Security Hub.



RESILIENCE

Multi-region active-active

DynamoDB Global Tables, Route 53 latency-based routing, automatic failover across regions.



IDENTITY

OAuth federation

Sign in with Google, GitHub, and Microsoft via Cognito identity providers. MFA enforcement per role.



OBSERVABILITY

X-Ray distributed tracing

Trace every request across the full pipeline. Latency breakdowns, cold-start visibility, dependency maps.



OPERATIONS

Infrastructure as Code

Terraform or AWS CDK for the full stack. CI/CD on push, automated regression tests, one-command teardown.

Thank you.

Questions, critiques, or attacks welcome.

*Never trust, **always verify***

THE TEAM

● Raj Desai

● Karan Vora

● Dhruv Shelke



github.com/Dhruvshelke15/zerowall